

ITAM

Instituto Tecnológico Autónomo de México

Diplomado en Artificial Intelligence & Large Language Models for Financial Markets

PROYECTO FINAL — MÓDULO 1

ML Pipeline End-to-End: Del Notebook al Servicio en Producción

Coordinador académico: **M.F. José Jorge Ramírez Olvera**

Instructor del Módulo 1: **Noé Oswaldo Rodríguez Rodríguez**

Módulo 1: *Advanced Python, ML Stack & LLM Tooling Foundations*

Fecha de presentación: **7 de mayo de 2026 (Clase 14)**

1. Descripción General

El proyecto final del Módulo 1 constituye un ejercicio integrador en el que cada estudiante pone en práctica la totalidad de las herramientas y conceptos cubiertos a lo largo de las 14 clases del módulo. El objetivo central consiste en construir un pipeline completo de Machine Learning —desde la obtención y validación de datos hasta el despliegue de una aplicación interactiva— empaquetado en un contenedor de Docker y servido desde la computadora del estudiante, de manera que el instructor y el resto del grupo puedan acceder al servicio de forma remota.

En términos concretos, se espera que cada estudiante elija un problema (preferentemente relacionado con finanzas, aunque se aceptan otros dominios como NLP, series temporales, clasificación de imágenes, etc.), entrene al menos dos modelos distintos para resolverlo, construya una aplicación en Streamlit que permita interactuar con las predicciones, y empaquete todo dentro de un contenedor de Docker que corra en su máquina local. El día de la presentación (7 de mayo de 2026), el instructor abrirá un navegador, escribirá la dirección IP y puerto del estudiante, y el servicio funcionará como si estuviera desplegado en la nube, salvo que estará corriendo en la laptop del estudiante.

El énfasis del proyecto no está en construir el modelo más sofisticado posible, sino en demostrar la capacidad de llevar una idea desde un notebook exploratorio hasta un entregable profesional de ingeniería de software: **código limpio, reproducible, testeado, versionado, documentado, validado y desplegado**. Se espera que el estudiante demuestre dominio de las prácticas de ingeniería de software para proyectos de ML que constituyen el eje central del Módulo 1.

2. Modalidad

El proyecto es estrictamente individual. Cada estudiante entrega su propio repositorio de GitHub con el historial completo de commits como evidencia de trabajo. Se permite la consulta entre compañeros y la discusión de enfoques generales, pero el código, la configuración, los tests y la dockerización constituyen trabajo personal e intransferible.

Es responsabilidad de cada estudiante verificar con anticipación que Docker funciona correctamente en su máquina, que es posible exponer puertos y que la conexión a la red que se utilice el día de la presentación permite que otros dispositivos se conecten al servicio.

3. El Problema a Resolver

El problema y el dataset son de libre elección. La única restricción es que el proyecto involucre datos tabulares, series temporales o texto (o alguna combinación de estos), y que tenga sentido entrenar un modelo de ML sobre esos datos. Se valora positivamente que el problema tenga alguna componente financiera, aunque no es obligatorio; lo determinante es que se aplique el stack completo del Módulo 1.

A continuación se listan algunas ideas orientativas:

- **Clasificación de sentimiento en noticias financieras.** Utilizar Financial PhraseBank o un corpus similar, entrenar un baseline con TF-IDF + LogisticRegression, y comparar contra un

modelo basado en embeddings de Hugging Face (FinBERT, sentence-transformers). La app en Streamlit recibe un texto y devuelve el sentimiento con probabilidades.

- **Predicción de dirección de precio.** Descargar datos de Yahoo Finance con yfinance, construir features temporales (lags, rolling, indicadores técnicos) y entrenar un clasificador que prediga si el precio sube o baja al día siguiente. La app recibe un ticker y muestra la predicción con gráficas históricas.
- **Scoring de riesgo crediticio.** Partir de un dataset público de crédito (Kaggle ofrece varias opciones robustas), entrenar un pipeline de scikit-learn y un MLP en PyTorch, y comparar ambos. La app recibe los datos de un solicitante y devuelve el score de riesgo con una explicación visual.
- **Detección de anomalías en métricas operacionales.** Simular o conseguir datos de series temporales (latencia, tráfico, costos), implementar detección basada en rolling windows y comparar contra un autoencoder en PyTorch. La app muestra un dashboard con las anomalías detectadas.
- **Clasificación de imágenes, recomendación, clustering u otro dominio.** Es válido proponer un problema fuera del ámbito financiero siempre y cuando se aplique el stack completo del Módulo 1. Ante dudas sobre la viabilidad de una idea, se recomienda consultar con el instructor antes de iniciar.

Lo determinante no es la complejidad del modelo ni la espectacularidad del dataset, sino la ejecución profesional del proyecto de punta a punta, con énfasis en las prácticas de ingeniería de software.

4. Requisitos Técnicos

A continuación se detallan los requisitos técnicos del proyecto, organizados por tema. Todos son obligatorios salvo que se indique explícitamente lo contrario. Se espera que el estudiante aplique la totalidad de lo cubierto en el Módulo 1.

4.1 Estructura de Proyecto y Python Avanzado

El repositorio debe seguir una estructura profesional tipo src-layout. No se aceptan notebooks sueltos ni scripts desorganizados en la raíz. La estructura mínima esperada es la siguiente:

```
mi-proyecto/
├── src/
│   ├── mi_paquete/
│   │   ├── __init__.py
│   │   ├── data/           # carga, validación, transformación
│   │   ├── models/         # definición y entrenamiento de modelos
│   │   ├── features/        # feature engineering
│   │   ├── evaluation/      # métricas y comparación
│   │   └── app/             # código de la app Streamlit
├── tests/                   # pytest
├── notebooks/               # exploración (NO código de producción)
├── data/                    # .gitignore, versionado con DVC
├── models/                  # artefactos serializados (.pkl, .pt)
├── configs/                 # YAML de configuración
├── Dockerfile
├── .dockerignore
├── pyproject.toml
├── .pre-commit-config.yaml
├── .gitignore
├── .env.example             # sin secretos reales
├── dvc.yaml
└── README.md
```

Dentro del código fuente se espera uso natural de las herramientas de Python avanzado cubiertas en las Clases 1 y 2:

- **Type hints** en todas las funciones públicas (parámetros y retorno). No se requiere mypy estricto, pero las firmas deben ser informativas.
- Al menos un **decorador con propósito real**: logging de tiempo de ejecución, retry, validación de inputs, caching, o cualquier otro que tenga sentido dentro del proyecto.
- Uso de **dataclasses** o **NamedTuple** para representar configuraciones, resultados de evaluación o cualquier estructura de datos con campos nombrados. No se aceptan diccionarios sueltos para datos con estructura fija.

- **Context managers** donde aplique: manejo de archivos, conexiones, recursos temporales. La serialización de modelos o escritura de artefactos debe realizarse dentro de un bloque with.
- **Configuración separada del código.** Los hiperparámetros, rutas y parámetros del modelo deben residir en archivos YAML o en variables de entorno gestionadas mediante python-dotenv, no hardcodeados dentro de las funciones.
- **Manejo seguro de secretos.** El repositorio debe incluir un archivo .env.example con las variables necesarias (sin valores reales). Las API keys y credenciales nunca deben aparecer en el código ni en el historial de commits.

4.2 Gestión de Dependencias y Calidad de Código

Se espera que el proyecto utilice Poetry o uv como gestor de dependencias. Es obligatorio contar con un pyproject.toml completo con todas las dependencias fijadas (versiones exactas o rangos acotados) y un lockfile generado. No se acepta un requirements.txt sin lockfile.

En cuanto a calidad de código:

- **ruff** configurado en pyproject.toml como linter y formateador. Las reglas específicas quedan a criterio del estudiante, pero deben estar explícitas en la configuración.
- **Pre-commit hooks** instalados y funcionales. Mínimo: ruff (lint + format), trailing whitespace, end-of-file fixer y check-yaml. Los hooks deben estar definidos en .pre-commit-config.yaml. Se verificará que el repositorio no contenga commits que los violen.
- **Logging en vez de prints.** Todo el código de producción (lo que reside en src/) debe usar el módulo logging de Python con niveles apropiados (INFO para flujo general, DEBUG para detalles, WARNING para situaciones recuperables, ERROR para fallas). Los notebooks pueden usar print; src/ no.

4.3 Orientación a Objetos y Diseño

Se espera que el proyecto demuestre uso deliberado de clases —no por mero cumplimiento, sino porque la abstracción resuelve un problema concreto de diseño. Requisitos mínimos:

- Al menos una **clase abstracta (abc.ABC)** que defina la interfaz común de los modelos. Por ejemplo, un BaseModel con métodos train() y predict(), del que hereden los modelos concretos (scikit-learn, PyTorch). Esto permite intercambiar modelos sin modificar el código que los consume.
- **Separación de responsabilidades:** la lógica de carga de datos no debe estar en la misma clase que el entrenamiento, y el entrenamiento no debe estar mezclado con la interfaz de Streamlit.
- Al menos un **patrón de diseño** aplicado con intención (Strategy para selección de modelos, Factory para crear pipelines, Repository para acceso a datos, u otro que resulte pertinente). Es necesario documentar brevemente en el README cuál se usó y por qué.

4.4 Stack Científico: NumPy, Pandas, Matplotlib

Todo el procesamiento de datos debe aprovechar las herramientas del stack científico vistas en el módulo:

- **NumPy** para operaciones numéricas vectorizadas. La presencia de loops de Python iterando sobre arrays para realizar cálculos aritméticos se considera una deficiencia.
- **Pandas** para carga, limpieza, transformación y exploración de datos tabulares o temporales. Se espera uso correcto de dtypes, manejo de NaN con estrategia documentada y, en caso de trabajar con fechas, DatetimeIndex con resample/rolling donde aplique.
- **Matplotlib (o Seaborn/Plotly)** para al menos 3 visualizaciones distintas dentro de la app de Streamlit o en el notebook exploratorio: una de distribución (histograma, boxplot), una de series temporales o tendencia, y una de comparación de modelos (barras, radar, heatmap de confusión). Todas las gráficas deben incluir títulos, etiquetas de ejes y leyendas.

4.5 Scikit-learn: Pipelines y Validación

Al menos uno de los dos modelos (o el baseline) debe ser un pipeline de scikit-learn que incluya:

- **Preprocesamiento integrado en el pipeline** (StandardScaler, OneHotEncoder u otro) mediante ColumnTransformer. No se acepta preprocesamiento separado que se omita al serializar el modelo.
- **Validación cruzada** con cross_val_score o estrategia equivalente. Si los datos son temporales, es obligatorio usar TimeSeriesSplit para evitar data leakage.
- Al menos **dos métricas de evaluación** apropiadas para el tipo de problema (clasificación: F1 + AUC-ROC; regresión: RMSE + R²; etc.). Las métricas deben reportarse en el README y en la app.
- **Búsqueda de hiperparámetros** (GridSearchCV o RandomizedSearchCV) con al menos 2 hiperparámetros explorados. Es necesario documentar qué se probó y qué se encontró.

4.6 Validación de Datos y Tracking de Experimentos

Estos dos componentes convierten un notebook exploratorio en un pipeline con garantías de producción:

- **Pandera:** Es obligatorio definir un DataFrameSchema que valide el dataset de entrada antes de que entre al pipeline de entrenamiento. El schema debe verificar al menos: tipos de columna, rangos válidos para las columnas numéricas y manejo de nulos. Si los datos no pasan la validación, el pipeline debe fallar con un mensaje claro, no continuar con datos corruptos.
- **MLflow:** Todos los experimentos de entrenamiento deben quedar registrados en MLflow. Cada run debe loggear: hiperparámetros (mlflow.log_param), métricas finales (mlflow.log_metric), y el modelo serializado como artefacto (mlflow.log_artifact o mlflow.sklearn.log_model / mlflow.pytorch.log_model). Al abrir la UI de MLflow debe ser posible ver la comparación entre todos los experimentos y entender las decisiones tomadas.

4.7 PyTorch: Modelo desde Cero

Al menos uno de los modelos del proyecto debe estar implementado en PyTorch (no en scikit-learn ni en una librería de alto nivel que abstraiga PyTorch). Requisitos:

- Una clase que herede de **nn.Module** con un forward() explícito. Puede ser un MLP, una red recurrente, un autoencoder, o lo que aplique al problema.
- **Dataset personalizado** (hereda de torch.utils.data.Dataset) con __len__ y __getitem__.
- **Training loop completo:** forward → loss → backward → optimizer.step() → zero_grad. Con separación de train/validation, logging de métricas por época, y early stopping o al menos checkpointing del mejor modelo.
- El modelo final debe estar serializado con torch.save() y debe poder cargarse para inferencia sin necesidad de reentrenar.

4.8 Hugging Face (Opcional pero Valorado)

No es obligatorio, pero si el problema involucra texto, se espera que se utilice Hugging Face de alguna forma: embeddings con sentence-transformers como features para el modelo, un modelo preentrenado de la librería transformers como baseline o como uno de los dos modelos a comparar, o la librería datasets para cargar un corpus del Hub. Si el problema es puramente numérico y no involucra texto, este requisito no aplica.

4.9 Git, GitHub y Versionado

El repositorio de GitHub es el **entregable principal**. Debe reflejar un flujo de trabajo profesional:

- **Branch main protegido.** No deben existir commits directos a main. Todo entra por pull request desde feature branches. El historial de main debe ser limpio y cada merge debe representar una funcionalidad completa.
- **Conventional commits.** Los mensajes de commit deben seguir la convención: feat: ..., fix: ..., docs: ..., refactor: ..., test: ..., chore: No se aceptan mensajes tipo 'asdfg', 'wip' o 'cambios varios'.
- **Pull requests con descripción.** Cada PR debe tener un título descriptivo y un cuerpo que explique qué cambia y por qué. No necesita ser extenso — 2-3 oraciones bastan — pero debe existir.
- **Pre-commit hooks activos.** Los hooks de ruff deben ejecutarse antes de cada commit. Si un commit llegó al repositorio con errores de lint, es evidencia de que los hooks no estaban instalados.
- **.gitignore robusto.** Datos, modelos binarios, archivos .env, __pycache__, .venv, outputs de notebooks y cualquier artefacto generado deben estar excluidos del repositorio.
- **GitHub Actions (opcional pero valorado).** Configurar un workflow de CI que ejecute los tests y el linter en cada PR. No es obligatorio, pero demuestra madurez en el flujo de trabajo y suma en la evaluación.

4.10 DVC: Versionado de Datos y Modelos

Los archivos pesados —datasets, modelos serializados, artefactos de MLflow— no pertenecen a Git. Para eso está DVC:

- dvc init en el repositorio, con al menos los datos y los modelos trackeados mediante dvc add.
- Un remote configurado (local, Google Drive, S3 u otro). Lo relevante es que al ejecutar dvc pull se descarguen los artefactos necesarios para reproducir el proyecto.
- Idealmente (no obligatorio): un dvc.yaml que defina el pipeline de etapas (ingest → preprocess → train → evaluate) y que se pueda reproducir con dvc repro.

4.11 Docker y Streamlit: El Despliegue

Este componente es el que distingue al proyecto de un notebook académico. Se construye una imagen de Docker que contiene la aplicación de Streamlit lista para servir predicciones. **Es la parte más importante desde la perspectiva de ingeniería de software.**

Dockerfile

El Dockerfile debe cumplir con lo siguiente:

- Imagen base razonable (python:3.11-slim o similar; no una imagen completa de 2 GB si no es necesaria).
- Copiar solo lo necesario, respetando el .dockerignore (excluir .git, data cruda, notebooks, .venv).
- Instalar dependencias desde el lockfile para garantizar reproducibilidad exacta.
- Incluir el modelo ya serializado (no reentrenar dentro del contenedor).
- Exponer el puerto de Streamlit (por defecto 8501).
- CMD o ENTRYPOINT que arranque la app de Streamlit.

Aplicación Streamlit

La app debe ser funcional y usable. No se evalúa el diseño visual, pero sí la funcionalidad:

- Una página principal que explique brevemente qué hace el modelo y cómo usarlo.
- Un formulario o mecanismo de input para que el usuario ingrese datos (campos de texto, sliders, file upload, según aplique al problema).
- La predicción del modelo mostrada de forma clara, con probabilidades o scores en caso de clasificación.
- Al menos una visualización generada dinámicamente (una gráfica de Matplotlib o Plotly que dependa del input del usuario).
- Una sección de '**Acerca del modelo**' que muestre las métricas de evaluación y una comparación breve de los modelos entrenados.

Servir desde la Computadora del Estudiante

El día de la presentación (7 de mayo de 2026), el flujo es el siguiente: el estudiante ejecuta docker build y docker run en su máquina, exponiendo el puerto 8501 (u otro de su elección), y comparte su dirección IP local y puerto con el grupo. Desde la computadora del instructor se accederá a la aplicación vía navegador en `http://<ip-estudiante>:8501`.

Es indispensable verificar con anticipación que la máquina permite conexiones entrantes en el puerto elegido y que existe conectividad dentro de la red. Se recomienda realizar una prueba al menos una semana antes de la fecha de presentación:

- **En Mac:** Configuración del Sistema → Red → Firewall.

- **En Windows:** Windows Defender Firewall → Reglas de entrada.
- **En Linux:** Verificar iptables o ufw.

Si la red del campus no permite tráfico entre dispositivos (aislamiento de clientes, frecuente en redes universitarias), la alternativa es utilizar ngrok o una herramienta equivalente para crear un túnel HTTP temporal que genere una URL pública apuntando al servicio local. Sin embargo, se recomienda intentar primero con la red local, por ser la opción más limpia y cercana a un entorno profesional.

5. EL README

El README.md es la carta de presentación del proyecto. No es un ensayo ni un paper, sino la documentación que cualquier persona necesita para entender y reproducir el trabajo. Contenido mínimo:

1. Título del proyecto y una descripción de una o dos oraciones sobre qué hace.
2. Nombre del estudiante y usuario de GitHub.
3. Instrucciones de instalación: cómo clonar, instalar dependencias con Poetry/uv, configurar variables de entorno y descargar datos con DVC.
4. Cómo ejecutar el proyecto: comandos exactos para entrenar el modelo, correr los tests y levantar la app (tanto con Docker como sin Docker, para desarrollo local).
5. Descripción del problema y del dataset: qué se resuelve, de dónde provienen los datos, qué variables contiene.
6. Resumen de resultados: métricas de los modelos entrenados, tabla comparativa y conclusión breve sobre cuál funciona mejor y por qué.
7. Arquitectura del proyecto: diagrama o descripción del flujo de datos (puede ser un esquema en ASCII o una imagen simple).
8. Decisiones de diseño: qué patrón de diseño se usó y por qué, cómo se estructuraron las clases, qué trade-offs se consideraron.

6. Rúbrica de Evaluación

La calificación se distribuye de la siguiente forma. Cada rubro se evalúa en una escala de 0 a 10, donde 0 es ausencia total, 5 es cumplimiento mínimo con deficiencias claras, y 10 es ejecución impecable.

Rubro	Peso	Qué se evalúa
Estructura y Python avanzado	10%	src-layout, type hints, decoradores, dataclasses, config separada, logging, manejo de secretos.
Dependencias y calidad de código	10%	pyproject.toml con lockfile, ruff configurado, pre-commit hooks funcionales.
OOP y diseño	10%	Clase abstracta, separación de responsabilidades, patrón de diseño documentado.
Stack científico	10%	Uso correcto de NumPy/Pandas, visualizaciones con contexto, feature engineering.
Scikit-learn pipeline	10%	Pipeline integrado, validación cruzada, búsqueda de hiperparámetros, métricas.
Pandera + MLflow	10%	Schema de validación, experimentos logueados, modelos registrados.
PyTorch	10%	nn.Module, Dataset, training loop, serialización.
Git y GitHub	10%	Branches, PRs, conventional commits, pre-commit hooks, .gitignore. CI es bonus.
DVC	5%	Datos y modelos versionados, remote configurado.
Docker + Streamlit	10%	Imagen funcional, app usable, servicio accesible el día de la presentación.
README y documentación	5%	Completo, claro, reproducible. Instrucciones que funcionan al seguirlas.

Nota: el uso de Hugging Face no tiene rubro separado, pero si el proyecto involucra texto y se utiliza correctamente, se refleja positivamente en los rubros de stack científico y PyTorch. Del mismo modo, GitHub Actions suma en el rubro de Git.

7. Entregables y Fecha de Entrega

7.1 Entregables

Se espera la entrega de los siguientes elementos:

1. URL del repositorio de GitHub (debe ser público o con acceso otorgado al instructor como colaborador).
2. El repositorio debe contener todo lo descrito en la sección 4: código fuente, tests, Dockerfile, configuración de pre-commit, pyproject.toml, dvc.yaml, README y los notebooks exploratorios.

3. Los datos y modelos deben ser descargables con `dvc pull`. Es responsabilidad del estudiante asegurar que el remote sea accesible para el instructor.
4. La imagen de Docker debe poderse construir con `docker build -t <nombre> .` sin modificaciones ni pasos manuales adicionales.

7.2 La Presentación

La presentación consta de dos partes:

- **Demo en vivo (10 minutos):** El estudiante levanta el contenedor en su máquina, comparte la URL, y el instructor accede a la aplicación desde su navegador. Durante la demo se espera una explicación breve de lo que muestra la app. También se revisará el repositorio de GitHub en pantalla: historial de commits, pull requests y estructura de archivos.
- **Preguntas (5 minutos):** El instructor realizará preguntas técnicas sobre decisiones de implementación. Ejemplos: '¿por qué esta arquitectura de red?', '¿qué ocurre si la validación de Pandera falla?', '¿cómo se maneja el caso de que el usuario ingrese datos fuera de rango en la app?', '¿por qué TimeSeriesSplit en vez de KFold?'. No se trata de memorizar, sino de demostrar comprensión de lo construido.

7.3 Fecha

El proyecto se entrega y presenta en la **última sesión del Módulo 1 (Clase 14), el día 7 de mayo de 2026**. El repositorio debe estar listo (último commit en main) al menos **24 horas antes** de la sesión de presentación para permitir una revisión anticipada por parte del instructor.

8. Guía de Ejecución Paso a Paso

La siguiente guía describe el orden recomendado para abordar el proyecto. No es un requisito seguir este orden al pie de la letra, pero sí refleja una secuencia lógica que minimiza retrabajo.

Fase 1 — Infraestructura (Semana 1)

- Crear el repositorio en GitHub con branch main protegido.
- Configurar la estructura src-layout completa con todos los directorios.
- Inicializar pyproject.toml con Poetry o uv e instalar las dependencias base.
- Configurar ruff en pyproject.toml y los pre-commit hooks en .pre-commit-config.yaml.
- Configurar .gitignore, .dockerignore y .env.example.
- Inicializar DVC con dvc init y configurar el remote.
- Realizar el primer commit a través de un pull request.

Fase 2 — Datos y Exploración (Semana 2)

- Obtener el dataset y agregarlo con dvc add.
- Crear el notebook exploratorio en notebooks/ con análisis de distribuciones, valores faltantes, correlaciones.
- Definir el DataFrameSchema de Pandera con las validaciones requeridas.
- Implementar las funciones de carga y preprocesamiento en src/ con type hints, logging y configuración YAML.

Fase 3 — Modelado (Semana 3)

- Implementar la clase abstracta BaseModel y los modelos concretos que hereden de ella.
- Construir el pipeline de scikit-learn con ColumnTransformer y validación cruzada.
- Implementar el modelo en PyTorch con nn.Module, Dataset personalizado y training loop.
- Registrar todos los experimentos en MLflow con parámetros, métricas y artefactos.
- Escribir los tests con pytest para las funciones críticas.

Fase 4 — App y Despliegue (Semana 4)

- Construir la aplicación de Streamlit con input, predicción, visualización e información del modelo.
- Escribir el Dockerfile y construir la imagen.
- Probar que el contenedor sirve la app correctamente en http://localhost:8501.
- Probar acceso desde otra máquina en la misma red.
- Completar el README con todos los elementos requeridos.
- Asegurar que el historial de Git refleja un flujo profesional con conventional commits y PRs.

9. Checklist de Entrega

Antes de entregar, se recomienda revisar la siguiente lista punto por punto. Si todos los elementos están cubiertos, el proyecto está listo.

Componente	Verificar que...
Repositorio	Repositorio de GitHub creado, público o con acceso para el instructor.
Estructura	src-layout con paquete Python, tests/, notebooks/, configs/, Dockerfile.
Python avanzado	Type hints, decorador, dataclass/NamedTuple, context managers, manejo de secretos.
Dependencias	pyproject.toml con Poetry o uv, lockfile presente.
Calidad	ruff + pre-commit configurados y funcionales. Logging en src/.
OOP	Clase abstracta, herencia, al menos un patrón de diseño documentado.
NumPy/Pandas	Operaciones vectorizadas, no loops sobre arrays. dtypes correctos.
Visualización	Al menos 3 gráficas con títulos, ejes, leyendas.
Scikit-learn	Pipeline con ColumnTransformer, CV, GridSearch, métricas.
Pandera	Schema definido, validación antes del entrenamiento.
MLflow	Experimentos logueados con params, métricas, y modelo.
PyTorch	nn.Module, Dataset custom, training loop, serialización.
Git	Branches, PRs, conventional commits, pre-commit hooks, .gitignore, no datos en Git.
DVC	Datos y modelos trackeados, remote configurado, dvc pull funcional.
Docker	Dockerfile, .dockerignore, imagen se construye, contenedor sirve la app.
Streamlit	App funcional con input, predicción, visualización, info del modelo.
README	Instalación, ejecución, descripción del problema, resultados, arquitectura, diseño.
Red	El servicio es accesible desde otra máquina (probado con anticipación).

10. Recomendaciones Finales

- **Comenzar por la infraestructura.** Conviene configurar la estructura del repositorio, el pyproject.toml, los pre-commit hooks, el .gitignore y la estructura de carpetas antes de escribir la primera línea de modelo. Reorganizar 40 archivos a última hora es un problema completamente evitable.
- **No subestimar Docker y Streamlit.** Los modelos suelen estar listos con relativa rapidez; lo que consume tiempo es hacer que todo funcione dentro de un contenedor: problemas de rutas, dependencias que faltan en la imagen, puertos que no se abren. Se recomienda reservar tiempo exclusivo para la dockerización e integración.
- **Probar la red con anticipación.** No esperar al día de la presentación para descubrir que el firewall bloquea conexiones entrantes o que la red del campus aísla dispositivos. Se recomienda levantar un contenedor con un 'Hello World' de Streamlit al menos una semana antes y verificar que otro dispositivo puede acceder.
- **Cuidar el README.** Un proyecto técnicamente bueno con un README pobre transmite desorden. Un proyecto correcto con un README claro y completo transmite profesionalismo.
- **Elegir un problema interesante.** Los mejores proyectos finales no son necesariamente los más complejos, sino aquellos en los que se nota interés genuino por el problema. Elegir un tema motivante se traduce en un mejor resultado.
- **Pensar en el proyecto como un producto de software.** La mentalidad que se busca no es la de un ejercicio académico, sino la de un servicio que otra persona va a usar. El instructor accederá al servicio como lo haría un usuario final: si no funciona, si no hay documentación o si la app no es comprensible, el proyecto no cumple su objetivo.

*Instructor: Noé Oswaldo Rodríguez Rodríguez
Coordinador académico: M.F. José Jorge Ramírez Olvera
ITAM — 2026*